

Securing your web app

Staying on top of the common vulnerabilities

Adhyayan Panwar | feedback@digit.in

Everyone and their grandmas are online now. And begone are the days of MySpace (of course), Facebook and other social networks and welcome the days of blogs and self-scripted websites. But as the great web hacker Spider-man once said, "With great web-coding powers, comes great security vulnerabilities". But hey, you can't afford those vulns now, can you?

So it's time to refill your web-shooters and shoot down those nasty bugs and loopholes sprawling through the beautiful website that you created to flex on your buddies and to ponder upon the harrowing realization of how vulnerable that website actually was.

THE BUGS:

Since everyone will be interacting with the website before anything else, that's where most of the attacks happen. Below are the major flaws that are most prominently present on a website:

Major website vulnerabilities:

Injection: When you created the website and meticulously coded it, you did it for the average user who will input data that you expect and that's where you lose the game. Because hackers will input anything and everything until the website gives in. These malicious inputs are called "injection". Injections can be targeted towards many services, most popularly SQL/NoSQL (databases), LDAP (a directory access protocol) and

OS injections. Let's look at how injections work. Say you use this query to retrieve authenticate a user with credentials "abc:pass123" from a database:

```
SELECT * FROM data-
base WHERE user='abc' AND
password='pass123'
```

If the query result is not empty (i.e, it returns some data. Since user and password combination is unique, it will fetch a row of data), the user is authenticated. Imagine someone put "OR 1=1 --" as username and "hacked" as a password. Let's take a look at the query now:

```
SELECT * FROM database
WHERE user=' ' OR 1=1 --' AND
password='hacked'
```

This changes everything. Because of the single quote, the user is simply a space character and "--" is used to denote a comment in MySQL, hence everything after that is disregarded by the SQL parser. And since 1 is always equal to 1, this query will always be executed! Hence, login is bypassed. This is a basic example of SQL injection, to make you understand how inputs can be crafted to exploit services. There are a huge amount of payloads present, which are more complex than the previous one.

Cross Site Scripting (XSS): Aha, the good old XSS. A kind of injection itself, JavaScript injection to be precise, but because of the enormous-ness of the attack, it's usually considered a category in itself. In XSS attacks, any input that might be displayed later or stored somewhere and retrieved is crafted in such a way that it fools the browser in executing malicious scripts. For example, in a comment section someone puts "<script>alert('Hacked!');</script>", next time someone visits the page, the browser will create a pop-up dialogue box saying "Hacked!" because it thought the comment was

something a developer put in (because of the <script></script> tags). But you might be thinking, adverse effects? Well, there certainly are. User sessions, maintained through cookies, can be stolen and can be used to impersonate them. Let's take it up a notch further. Assume there's a website where comments are first approved by an admin. So there's a comment "". Let us explain what's actually happening with this payload. Basically, an image is being inserted with the parameter that will execute the URL if there's an error in loading the image (the onerror parameter). Since the image source is wrong, the payload makes sure the URL is executed. That last part is what makes it interesting, "document.cookie" will append the cookie that was created when admin logged in, to the URL. A simple HTTP server can be used to log the info. And the admin account is compromised. XSS is categorised into three categories:

- **Reflected XSS:** The simplest form of XSS, where the input payload is executed immediately and might not affect others. The payload is usually injected in an HTTP request. For example, say that the given search URL is vulnerable to XSS: <https://insecure-website.com/search?string=Hello>, and the output is something like <p>Hello</p> but instead if you send the request [https://insecure-website.com/search?string=<script>alert\(1\);</script>](https://insecure-website.com/search?string=<script>alert(1);</script>) then the response will be immediately visible (reflected).
- **Stored XSS:** In Stored XSS, also known as persistent XSS, the injected payload is either stored somewhere in a database that might be reflected on a later stage or the payload is embedded into subsequent HTTP requests the webpage makes, hence effectively penetrating the server-side. The scenario we talked about where the admin account was compromised was a case of stored XSS.
- **DOM-based XSS:** A DOM (Document Object Model) in JavaScript is a tree-like structure according to which objects of a webpage are stored.